



Sinfama

Caja de compensacion

Agente SQL - Procesos RPA

Guía de Instalacion y Ejecucion

Este documento describe como clonar, configurar y ejecutar el proyecto de analisis de procesos RPA con agente SQL local (Ollama + Qwen 2.5 Coder), entrada por voz (Whisper) y calculadora deterministica de ROI en cualquier equipo con Windows, macOS o Linux.

1. Requisitos previos

Herramienta	Version minima	Descarga
Git	2.30+	https://git-scm.com
Git LFS	3.0+	https://git-lfs.github.com
uv	0.4+	https://docs.astral.sh/uv
Python	3.11+	https://python.org/downloads
Ollama	0.23+	https://ollama.com/download

Nota: GPU NVIDIA es opcional. Ollama detecta CUDA y hace fallback a CPU automaticamente. uv puede instalar Python 3.11 por ti si aun no lo tienes.

2. Instalar Git LFS (obligatorio para las bases de datos)

El repositorio usa Git LFS para los archivos .db (bases de datos SQLite, ~300 MB). Sin esto los archivos quedaran vacios.

```
# Windows (winget)
winget install GitHub.GitLFS

# macOS
brew install git-lfs

# Ubuntu / Debian
sudo apt install git-lfs

# Activar globalmente (una sola vez por maquina)
git lfs install
```

3. Clonar el repositorio

```
git clone https://github.com/miloguz/Proyecto1Especializacion.git
cd Proyecto1Especializacion

# Si los .db quedaron vacios:
git lfs pull
```

4. Instalar uv (si no lo tienes)

uv es el gestor de entornos y dependencias del proyecto. Si aun no esta instalado en tu maquina, ejecuta:

```
pip install uv
```

Nota: Alternativa oficial (instalador independiente de Python): <https://docs.astral.sh/uv/>

5. Instalar dependencias con uv

Crea un entorno virtual aislado en .venv/ e instala todas las dependencias declaradas en pyproject.toml / uv.lock:

```
uv sync
```

6. Instalar Ollama y descargar el modelo LLM

El agente SQL necesita Ollama corriendo localmente y el modelo qwen2.5-coder:7b (~4.7 GB).

```
# Terminal 1 - servidor Ollama
ollama serve

# Terminal 2 - descargar modelo (solo la primera vez)
ollama pull qwen2.5-coder:7b
```

Nota: En Windows, Ollama se instala como servicio y arranca automaticamente. No es necesario 'ollama serve' de forma manual.



7. Descargar el modelo Whisper (entrada por voz, ~480 MB)

Whisper-small transcribe la voz a texto localmente y habilita el botón de micrófono en la pestaña de Chat SQL. Sin este paso el botón aparece pero la transcripción falla al usarlo.

```
uv run python scripts/download_whisper.py
```

Nota: Solo es necesario una vez por máquina. El modelo queda cacheado en ~/.cache/huggingface/ y se reutiliza en todas las ejecuciones futuras.

8. (Opcional) Entrenar el modelo experimental de ROI

La BD limpia y los datos crudos vienen incluidos vía Git LFS, así que la app puede correr inmediatamente después del paso 7. Este paso solo es necesario si quieres explorar el modelo predictivo en notebooks.

```
# Opcion A - pipeline reproducible
uv run python scripts/train_roi_model.py

# Opcion B - notebooks paso a paso
uv run jupyter lab
```

Nota: El modelo XGBoost (notebook 03 y train_roi_model.py) es exploratorio y NO está integrado en la app. La Calculadora de ROI usa exclusivamente la fórmula determinística.

9. Ejecutar la aplicación

```
# Opcion A - script rapido (solo Windows)
run_app.bat

# Opcion B - manual (Windows / macOS / Linux)
uv run streamlit run app/chat.py
```

Abrir en el navegador: <http://localhost:8501>

10. Solución de problemas comunes

Sintoma	Causa probable	Solución
Bases de datos vacías (0 bytes)	Git LFS no instalado	git lfs install && git lfs pull
ModuleNotFoundError al correr	Deps no instaladas	uv sync en la raíz del proyecto
'Ollama no está corriendo'	Servidor Ollama apagado	ollama serve en terminal aparte
Botón de micrófono no transcribe	Modelo Whisper no descargado	Ejecutar el paso 7
Ollama no usa la GPU	CUDA Toolkit no instalado	Instalar CUDA; fallback CPU automático

11. Estructura del proyecto

```

Proyecto1Especializacion/
  app/chat.py          <- Streamlit: Chat SQL / ROI / Calculadora
  assets/logo.svg     <- logo Sinfama
  src/
    agent/             <- agente SQL + conexión DB
    models/roi_predictor.py <- (experimental) XGBoost
    utils/roi_calculator.py <- calculo determinístico de ROI
  data/
    raw/               <- CSVs originales (Git LFS)
    database/Procesos_clean.db <- BD limpia (Git LFS)
  notebooks/
    01_preprocesamiento.ipynb <- limpieza y normalización
    02_eda_roi.ipynb         <- EDA enfocado en ROI
    03_modelo_roi.ipynb      <- (experimental) entrenamiento XGBoost
  scripts/
    csv_to_sqlite.py        <- CSVs -> Procesos.db
    train_roi_model.py      <- (experimental) pipeline XGBoost
    download_whisper.py     <- pre-descarga modelo whisper
  models/                <- (experimental) roi_model.joblib
  run_app.bat             <- acceso rápido Windows
  pyproject.toml          <- dependencias del proyecto

```

12. Fórmula de cálculo de ROI

```

Beneficio_Bruto = TiempoManual x ValorHora x NumEjecuciones
Costo_Robot    = DuracionRobot x 7300 x NumEjecuciones
Ahorro_Neto    = Beneficio_Bruto - Costo_Robot
ROI%           = (Ahorro_Neto / Costo_Robot) x 100

```

La pestaña 'Calculadora de ROI' de la app usa exclusivamente esta fórmula determinística, auditable y sin caja negra.

El costo operativo del robot se estandariza en 7.300 COP/hora para todas las soluciones del portafolio. Este valor proviene de la infraestructura real:

- Servidor Azure: 150.000 COP/mes
- Licencia UiPath robot + orquestador: 5.200.000 COP/mes
- Total mensual: 5.350.000 COP / ~730 h ≈ 7.300 COP/h

Aplicar una tarifa uniforme hace comparables los ROIs entre proyectos sin importar el rol que cada bot reemplace.

El modelo predictivo experimental (XGBoost, notebook 03) aplica transformación $\log_{1p}(\text{ROI})$ antes de entrenar para manejar la distribución sesgada del target. NO está integrado en la app.